What was my thinking?

Clojure code

The tasks seems simple

Until you consider the edge cases

This is where the real issues lie

Constraints

I won't do some validation of the radar data, I'll always assume that the data will be valid

Also, I feel figuring out the invader scans seems harder than data validation, hence this decision

What was easy? What was hard?

Scanning the radar for the invaders where the whole invader fits in the radar, that was fine.

Handling the edge cases was where this tast provides some "head scratching".

How did I handle the edge cases? The initial thinking.

At first I assumed that I'll just "cut" the "frame": grab the edges of radar and scan those for the invaders. That proved to be too ridgid.

How did I handle the edge cases? My final approach.

So I went with scanning the invaders with part of their body "hanging off" the radar. That is why I worked on adding the visible part of the invader, so that you can know how much hanging off to allow and consider for scanning.

What I didn't do

I do all the rotations of the invaders.

But when it comes to scanning of the corners, I didn't cover that.

I mean they are covered, but they are a different edge case. I figure I'll handle that in Ruby.

How I worked, part 1

(Why will I explain this? Because you also do Ruby as well as Clojure. In Ruby TDD is the default. Whereas in Clojure it isn't. So setting some context is relevant.)

I did REPL driven development, where I explored the data function by function, in Emacs CIDER.

How I worked, part 2

I explored/developed all in one namespace. When I got to a point where I felt confident that the solution made sense, I added tests to prevent regression.

Only when I saw what the solution could look like did I decide on splitting the functions in separate namespaces.